

BackendPatternConfig#

https://pytorch.org/docs/stable/generated/torch.quantization.backend_config

Description:

Config object that specifies quantization behavior for a given operator pattern. For a detailed example usage, see BackendConfig. Add a set of supported data types passed as arguments to quantize ops in the reference model spec. Create a BackendPatternConfig from a dictionary with the following items: “pattern”: the pattern being configured “observation_type”: the ObservationType that specifies how observers should be inserted for this pattern “dtype_configs”: a list of dictionaries that represents DTypeConfig s “root_module”: a torch.nn.Module that represents the root for this pattern “qat_module”: a torch.nn.Module that represents the QAT implementation for this pattern “reference_quantized_module”: a torch.nn.Module that represents the reference quantized implementation for this pattern’s root module. “fused_module”: a torch.nn.Module that represents the fused implementation for this pattern “fuser_method”: a function that specifies how to fuse the pattern for this pattern “pattern_complex_format”: the pattern specified in the reversed nested tuple format (deprecated)

Function Signature

```
class
torch.ao.quantization.backend_config.BackendPatternConfig(pa
ttern=None)[source]#
add_dtype_config(dtype_config)[source]#
Return type
classmethod from_dict(backend_pattern_config_dict)[source]#
Return type
```

Returns:

dict[str, Any]

Detailed Documentation

Documentation

Config object that specifies quantization behavior for a given operator pattern. For a detailed example usage, see BackendConfig.

Add a set of supported data types passed as arguments to quantize ops in the reference model spec.

Create a BackendPatternConfig from a dictionary with the following items:

“pattern”: the pattern being configured “observation_type”: the `ObservationType` that specifies how observers should be inserted for this pattern “dtype_configs”: a list of dictionaries that represents `DTypeConfig` s “root_module”: a `torch.nn.Module` that represents the root for this pattern “qat_module”: a `torch.nn.Module` that represents the QAT implementation for this pattern “reference_quantized_module”: a `torch.nn.Module` that represents the reference quantized implementation for this pattern’s root module. “fused_module”: a `torch.nn.Module` that represents the fused implementation for this pattern “fuser_method”: a function that specifies how to fuse the pattern for this pattern “pattern_complex_format”: the pattern specified in the reversed nested tuple format (deprecated)

Set the supported data types passed as arguments to quantize ops in the reference model spec, overriding all previously registered data types.

Set the module that represents the fused implementation for this pattern.

Set the function that specifies how to fuse this `BackendPatternConfig`’s pattern.

The first argument of this function should be `is_qat`, and the rest of the arguments should be the items in the tuple pattern. The return value of this function should be the resulting fused module.

For example, the fuser method for the pattern (`torch.nn.Linear`, `torch.nn.ReLU`) can be:

```
return torch.ao.nn.intrinsic.LinearReLU(linear, relu)
```

For a more complicated example, see <https://gist.github.com/jerryzh168/8bea7180a8ba3c279f2c9b050f2a69a6>.

Set how observers should be inserted in the graph for this pattern.

Observation type here refers to how observers (or quant-dequant ops) will be placed in

the graph. This is used to produce the desired reference patterns understood by the backend. Weighted ops such as linear and conv require different observers (or quantization parameters passed to quantize ops in the reference model) for the input and the output.

There are two observation types:

`OUTPUT_USE_DIFFERENT_OBSERVER_AS_INPUT` (default): the output observer instance will be different from the input. This is the most common observation type.

`OUTPUT_SHARE_OBSERVER_WITH_INPUT`: the output observer instance will be the same as the input. This is useful for operators like cat.

Note: This will be renamed in the near future, since we will soon insert QuantDeQuantStubs with observers (and fake quantizes) attached instead of observers themselves.

Set the pattern to configure.

The pattern can be a float module, functional operator, pytorch operator, or a tuple combination of the above. Tuple patterns are treated as sequential patterns, and currently only tuples of 2 or 3 elements are supported.

Set the module that represents the QAT implementation for this pattern.

Set the module that represents the reference quantized implementation for this pattern's root module.

For more detail, see `set_root_module()`.

Set the module that represents the root for this pattern.

When we construct the reference quantized model during the convert phase, the root

modules (e.g. `torch.nn.Linear` for `torch.ao.nn.intrinsic.LinearReLU`) will be swapped to the corresponding reference quantized modules (e.g. `torch.ao.nn.reference.quantized.Linear`). This allows custom backends to specify custom reference quantized module implementations to match the numerics of their lowered operators. Since this is a one-to-one mapping, both the root module and the reference quantized module must be specified in the same `BackendPatternConfig` in order for the conversion to take place.

Convert this `BackendPatternConfig` to a dictionary with the items described in `from_dict()`.